实现 EventEmitter 类，包含 on、once、emit、remove、removeAll

```
class EventEmitter {
    constructor() {
        this.events = {}
    }

    on(eventName, eventHandler) {
        if (!this.events[eventName]) {
            this.events[eventName] = [];
        }
        this.events[eventName].push(eventHandler)
    }

    once(eventName, eventHandler) {
        const onceHandler = (...args) => {
            eventHandler(...args)
            this.remove(eventName, onceHandler)
        }
        this.on(eventName, onceHandler)
    }

    emit(eventName, ...args) {
        if (this.events[eventName]) {
            for (const fn of this.events[eventName]) {
                fn(...args)
            }
        }
    }

    remove(eventName, eventHandler) {
        if (this.events[eventName]) {
            this.events[eventName] = this.events[eventName].filter(fn => fn !==
eventHandler)
        }
    }

    removeAll(eventName) {
        if (this.events[eventName]) delete this.events[eventName]
    }
}
```

```

//--------------以下为验证方法---------------

const eventEmitter = new EventEmitter()
eventEmitter.on("event1", (...args) => {
    console.log("event1", ...args)
})
eventEmitter.once("event2", (...args) => {
    console.log("event2", ...args)
})
eventEmitter.emit("event1", "你", "好", "呀")
eventEmitter.emit("event1", "你", "好", "呀2")
eventEmitter.emit("event2", "你", "好", "呀")
eventEmitter.emit("event2", "你", "好", "呀2")

const fn = () => {
    console.log("event3")
}
eventEmitter.on("event3", fn)
eventEmitter.emit("event3")
eventEmitter.remove("event3", fn)
eventEmitter.emit("event3")

eventEmitter.on("event4",() => {
    console.log("event4")
})
eventEmitter.emit("event4")
eventEmitter.removeAll("event4")
eventEmitter.emit("event4")
```

防抖、节流函数

```
//防抖：点击后延迟触发，延迟过程再次点击则重新计时
function debounce(fn, time) {
    let timer;
    return function (...args) {
        if (timer) clearTimeout(timer)
        timer = setTimeout(() => {
            fn.call(this, ...args);
```

```
        }, time)
    }
}


//节流：触发后需要冷却时间才能再次触发
function throttle(fn, time) {
    let flag = false;
    return function (...args) {
        if (flag) return;
        fn.call(this, ...args);
        flag = true
        setTimeout(() => {
            flag = false
        }, time)
    }
}

//hooks版本
function useDebounce(fn, timeout) {
    const timerRef = useRef(null);
    useEffect(() => {
        return () => {
            if (timerRef.current) clearTimeout(timerRef.current)
        }
    }, [])
    return useCallback((...args) => {
        if (timerRef.current) clearTimeout(timerRef.current);
        timerRef.current = setTimeout(() => {
            fn.call(this, ...args);
        }, timeout);
    }, [fn, timeout]);
}

function useThrottle(fn, timeout) {
    const flagRef = useRef(false)
    const timerRef = useRef(null)
    useEffect(() => {
        return () => {
            if (timerRef.current) clearTimeout(timerRef.current)
        }
    }, [])
```

```
    return useCallback((...args) => {
        if (flagRef.current) return;
        fn.call(this, ...args);
        flagRef.current = true;
        timerRef.current = setTimeout(() => {
            flagRef.current = false;
        }, timeout)
    }, [fn, timeout])
}
```

深克隆

```
function deepClone(obj, hash = new WeakMap()) {
    if (obj === null) return null;
    if (obj instanceof Date) return new Date(obj)
    if (obj instanceof RegExp) return new RegExp(obj)

    if (hash.has(obj)) return hash.get(obj)  //防止嵌套、拷贝自身
    if (typeof obj === "object") {
        let cloneObj = Array.isArray(obj) ? [] : {}
        for (const [key, value] of Object.entries(obj)) {
            cloneObj[key] = deepClone(value, hash)
        }
        hash.set(obj, cloneObj)  //防止嵌套、拷贝自身
        return cloneObj
    } else {
        return obj
    }
}
```

函数的柯里化

```
const getSum = (a, b, c) => a + b + c

function curry(func) {
    return function curriedFn(...args){
        if (args.length < func.length){
```

```
            return function (...newArgs){
                return curriedFn(...args,...newArgs)
            }
        }else {
            return func(...args)
        }
    }
}

const curriedGetSum = curry(getSum)
console.log(curriedGetSum(1, 2, 3))  //6
console.log(curriedGetSum(1)(2)(3))  //6
console.log(curriedGetSum(1, 2)(3))  //6
console.log(curriedGetSum(1)(2, 3))  //6
```

冒泡排序、快速排序

```
//冒泡排序
let arr = [3, 5, 2, 1, 6, 4];
const length = arr.length
for (let i = 0; i < length - 1; i++){
  for (let j = i + 1; j < length; j++){
    if (arr[i] < arr[j]) [arr[i],arr[j]] = [arr[j],arr[i]]
  }
}

//快速排序
function quickSort(arr) {
    if (arr.length <= 1) return arr
    const midIndex = Math.floor(arr.length / 2);
    const midValue = arr.splice(midIndex, 1)[0]  //注意这里是剪切
    const leftArr = [];
    const rightArr = [];
    for (const num of arr) {
        if (num > midValue) {
            rightArr.push(num)
        } else {
            leftArr.push(num)
        }
```

```
    }
    return [...quickSort(leftArr), midValue, ...quickSort(rightArr)]
}
```

call、apply、bind

```
//call
Function.prototype.call2 = function (context = window, ...args) {
    if (!(typeof context === "object" && context !== null)) {
        context = new Object(context)
    }
    const fnKey = Symbol()
    context[fnKey] = this
    const result = context[fnKey](...args)
    delete context[fnKey]
    return result
}

//apply
Function.prototype.apply2 = function (context = window, args) {
    if (!(typeof context === "object" && context !== null)) {
        context = new Object(context)
    }
    const fnKey = Symbol()
    context[fnKey] = this
    const result = context[fnKey](...args)
    delete context[fnKey]
    return result
}

//apply
Function.prototype.bind2 = function (thisArg, ...args) {
    const thisFn = this;
    return function (...newArgs) {
        return thisFn.apply(thisArg, [...args, ...newArgs])
    }
}

Function.prototype.bind2 = function (context, ...args) {
```

```
    const self = this;
    const fBound = function (...innerArgs) {
        // 当作为构造函数时，this 指向实例，此时结果为 true，将绑定函数的 this 指向该实例
        // 当作为普通函数时，this 指向 context，此时结果为 false
        return self.apply(this instanceof fBound ? this : context, args.concat(innerArgs));
    };
    // 使用 Object.create 创建一个新对象，其原型指向原函数的原型
    fBound.prototype = Object.create(this.prototype);
    return fBound;
};
```

Promise方法

```
Promise.all2 = function (list) {
    return new Promise((resolve, reject) => {
        const len = list.length;
        if (len === 0) {
            resolve([]);
            return;
        }
        const resolvedArr = [];
        let resolvedCount = 0;
        list.forEach((p, index) => {
            Promise.resolve(p).then(res => {
                resolvedArr[index] = res;
                resolvedCount++;
                if (resolvedCount === len) {
                    resolve(resolvedArr);
                }
            }, reject);
        });
    });
};

Promise.allSettled2 = function (arr) {
    return new Promise(resolve => {
        let results = []
        let settledCount = 0;
        arr.forEach((p, index) => {
```

```
            Promise.resolve(p).then(
                value => {
                    results[index] = {status: "fulfilled", value: value};
                    settledCount++; // 无论解决还是拒绝，都增加计数器
                    if (settledCount === arr.length) {
                        resolve(results); // 当所有promises都解决时，解析整个结果数组
                    }
                },
                reason => {
                    results[index] = {status: "rejected", reason: reason};
                    settledCount++; // 无论解决还是拒绝，都增加计数器
                    if (settledCount === arr.length) {
                        resolve(results); // 当所有promises都解决时，解析整个结果数组
                    }
                }
            );
        });
    });
};


Promise.any2 = function(list) {
  return new Promise((resolve, reject) => {
    for (let item of list) {
      item.then(res => resolve(res), err => {});
    }
  });
};

Promise.race2 = function (list) {
    return new Promise((resolve, reject) => {
        for (const p of list) {
            Promise.resolve(p).then(resolve, reject)
        }
    })
}
```

手绘图形

```
/*三角*/
.one{
    height: 0;
    width: 0;
    border: 100px solid transparent;
    border-left-color: red;
}

/* 半圆 */
.half-circle {
    background: red;
    width: 100px;
    height: 50px;
    border-top-left-radius: 50px;
    border-top-right-radius: 50px;
}

/* 扇形 */
.sector {
    background: red;
    width: 100px;
    height: 100px;
    border-top-left-radius: 100px;
}
```